SEV 인증서 관리

개요

AMD SEV 인증서 관리는 보안 가상화 환경에서 신뢰 체인을 구축하고 유지하는 핵심 기능입니다. SEV는 계층적 인증서 체인을 통해 플랫폼의 신뢰성을 보장하고, 게스트 VM과 호스트 간의 안전한 통신을 가능하게 합니다.

SEV 인증서 체인 구조 (ARK → ASK → CEK → PEK)

인증서 체인 개요

SEV 인증서 체인은 다음과 같은 계층 구조를 가집니다:

```
AMD Root Key (ARK)

↓ 서명

AMD Signing Key (ASK)

↓ 서명

Chip Endorsement Key (CEK)

↓ 서명

Platform Endorsement Key (PEK)

↓ 서명

Platform Diffie-Hellman (PDH)
```

각 인증서의 역할

1. AMD Root Key (ARK)

- **역할**: AMD의 최상위 루트 인증서
- **특징**: AMD에서 직접 관리하는 자체 서명 인증서
- **용도**: ASK 인증서의 서명 및 검증

2. AMD Signing Key (ASK)

- **역할**: AMD의 서명 키
- **특징**: ARK로 서명된 중간 인증서
- **용도**: CEK 인증서의 서명 및 검증

3. Chip Endorsement Key (CEK)

- **역할**: 특정 칩(CPU)의 인증 키
- **특징**: ASK로 서명된 칩별 고유 인증서
- **용도**: PEK 인증서의 서명 및 검증

4. Platform Endorsement Key (PEK)

- **역할**: 플랫폼의 기본 암호화 키
- **특징**: CEK로 서명되거나 OCA로 서명될 수 있음
- **용도**: 플랫폼 소유권 설정 및 PDH 서명

5. Platform Diffie-Hellman (PDH)

- **역할**: 키 교환을 위한 공개키
- **특징**: PEK로 서명된 임시 키 쌍
- **용도**: 보안 세션 설정 시 키 교환

인증서 타입 정의

```rust
use sev::certs::sev::Usage;


// 인증서 사용 용도 정의
let oca_usage = Usage::OCA; // Owner Certificate Authority
let ark_usage = Usage::ARK; // AMD Root Key
let ask_usage = Usage::ASK; // AMD Signing Key
let cek_usage = Usage::CEK; // Chip Endorsement Key
let pek_usage = Usage::PEK; // Platform Endorsement Key
let pdh_usage = Usage::PDH; // Platform Diffie-Hellman


println!("OCA 사용 용도: {}", oca_usage);
println!("PEK 사용 용도: {}", pek_usage);
```

인증서 검증 프로세스

1. 전체 체인 검증

```rust
use sev::certs::sev::{Chain, Verifiable};
use sev::firmware::host::Firmware;


let mut firmware = Firmware::open()?;


// 인증서 체인 내보내기
```

```
let  chain = firmware.pdh_cert_export()?;


// 인증서 체인 검증
match  chain.verify() {
Ok(verified_cert) => {
println!("인증서 체인 검증 성공");
println!("검증된 인증서: {:?}", verified_cert);
}
Err(e) => {
println!("인증서 체인 검증 실패: {:?}", e);
}
}
```

2. 개별 인증서 검증

```
use sev::certs::sev::{sev::Certificate, Verifiable};


fn  verify_individual_certificate(cert: &Certificate) -> Result<(), Box<dyn  std
// 인증서 기본 정보 확인
println!("인증서 버전: {:?}", cert.version());
println!("인증서 사용 용도: {:?}", cert.usage());
// 인증서 서명 검증 (상위 인증서 필요)
// 실제 검증은 상위 인증서와 함께 수행
```

```
Ok(())
}
```

3. 인증서 체인 구축 및 검증

```
use sev::certs::sev::{Chain, ca, sev};

fn build_and_verify_chain() -> Result<(), Box<dyn std::error::Error>> {
// CA 체인 구축 (ARK → ASK)
let ca_chain = ca::Chain {
ark: get_ark_certificate()?,
ask: get_ask_certificate()?,
};
// SEV 체인 구축 (CEK → PEK → PDH)
let sev_chain = sev::Chain {
cek: get_cek_certificate()?,
pek: get_pek_certificate()?,
pdh: get_pdh_certificate()?,
};
// 전체 체인 구축
let full_chain = Chain {
ca: ca_chain,
```

```
sev: sev_chain,
};
// 체인 검증
let verified = full_chain.verify()?;
println!("체인 검증 완료: {:?}", verified);
Ok(())
}
```

키 관리 및 암호화 작업

1. 키 생성

```
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// PEK 생성
firmware.pek_generate()?;
println!("PEK 생성 완료");

// PDH 키 쌍 생성
firmware.pdh_generate()?;
println!("PDH 키 쌍 생성 완료");
```

2. 개인키를 사용한 인증서 서명

```rust
use sev::certs::sev::{sev::Certificate, sev::Usage, Signer, PrivateKey};

// OCA 인증서와 개인키 생성
let (mut oca_cert, oca_private_key) = Certificate::generate(Usage::OCA)?;

// 자체 서명
oca_private_key.sign(&mut oca_cert)?;
println!("OCA 인증서 자체 서명 완료");

// PEK CSR 서명
let mut firmware = Firmware::open()?;
let mut pek_cert = firmware.pek_csr()?;

// OCA 개인키로 PEK 서명
oca_private_key.sign(&mut pek_cert)?;
println!("PEK 인증서 서명 완료");
```

3. 키 저장 및 로드

```rust
use std::fs;
use sev::certs::sev::sev::Certificate;
```

```
// 인증서를 파일로 저장
fn save_certificate(cert: &Certificate, filename: &str) -> Result<(), Box<dyn
let cert_data = cert.encode_to_vec()?;
fs::write(filename, cert_data)?;
println!("인증서가 {}에 저장되었습니다", filename);
Ok(())
}

// 파일에서 인증서 로드
fn load_certificate(filename: &str) -> Result<Certificate, Box<dyn std::error:
let cert_data = fs::read(filename)?;
let cert = Certificate::decode_from_slice(&cert_data)?;
println!("인증서가 {}에서 로드되었습니다", filename);
Ok(cert)
}
```

인증서 내보내기/가져오기 코드 예제

1. 인증서 체인 내보내기

```
use sev::firmware::host::Firmware;
use std::fs;
```

```rust
fn  export_certificate_chain() -> Result<(), Box<dyn  std::error::Error>> {

let  mut  firmware = Firmware::open()?;

// 전체 인증서 체인 내보내기

let  chain = firmware.pdh_cert_export()?;

// 각 인증서를 개별 파일로 저장

save_certificate(&chain.pdh, "pdh_cert.der")?;

save_certificate(&chain.pek, "pek_cert.der")?;

save_certificate(&chain.oca, "oca_cert.der")?;

save_certificate(&chain.cek, "cek_cert.der")?;

println!("모든 인증서가 성공적으로 내보내졌습니다");

// 체인 정보 출력

println!("=== 인증서 체인 정보 ===");

println!("PDH 사용 용도: {}", chain.pdh.usage());

println!("PEK 사용 용도: {}", chain.pek.usage());

println!("OCA 사용 용도: {}", chain.oca.usage());

println!("CEK 사용 용도: {}", chain.cek.usage());

Ok(())

}


fn  save_certificate(cert: &sev::certs::sev::sev::Certificate, filename: &str) -

// 인증서를 DER 형식으로 인코딩

let  encoded = cert.encode_to_vec()?;
```

```
fs::write(filename, encoded)?;

Ok(())

}
```

2. 인증서 가져오기

```
use sev::firmware::host::Firmware;

use sev::certs::sev::sev::Certificate;

use std::fs;


fn import_certificates() -> Result<(), Box<dyn std::error::Error>> {

let mut firmware = Firmware::open()?;

// 파일에서 인증서 로드

let pek_cert = load_certificate("pek_cert.der")?;

let oca_cert = load_certificate("oca_cert.der")?;

// 인증서를 플랫폼에 가져오기

firmware.pek_cert_import(&pek_cert, &oca_cert)?;

println!("인증서 가져오기 완료");

// 가져오기 검증

let imported_chain = firmware.pdh_cert_export()?;

if imported_chain.pek == pek_cert && imported_chain.oca == oca_cert {

println!("인증서 가져오기 검증 성공");

} else {
```

```
println!("인증서 가져오기 검증 실패");
}
Ok(())
}

fn load_certificate(filename: &str) -> Result<Certificate, Box<dyn std::error:
let cert_data = fs::read(filename)?;
let cert = Certificate::decode_from_slice(&cert_data)?;
Ok(cert)
}
```

3. 인증서 정보 조회

```
use sev::certs::sev::sev::Certificate;

fn inspect_certificate(cert: &Certificate) {
println!("=== 인증서 정보 ===");
println!("버전: {:?}", cert.version());
println!("사용 용도: {}", cert.usage());
// 공개키 정보
if let Ok(public_key) = cert.public_key() {
println!("공개키 알고리즘: {:?}", public_key.algorithm());
}
```

```
// 서명 정보
if let Ok(signature) = cert.signature() {
println!("서명 알고리즘: {:?}", signature.algorithm());
}
}

fn inspect_certificate_chain() -> Result<(), Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
let chain = firmware.pdh_cert_export()?;
println!("=== PDH 인증서 ===");
inspect_certificate(&chain.pdh);
println!("\n=== PEK 인증서 ===");
inspect_certificate(&chain.pek);
println!("\n=== OCA 인증서 ===");
inspect_certificate(&chain.oca);
println!("\n=== CEK 인증서 ===");
inspect_certificate(&chain.cek);
Ok(())
}
```

고급 인증서 관리

1. 인증서 갱신

```rust
use sev::firmware::host::Firmware;

use sev::certs::sev::{sev::Certificate, sev::Usage, Signer};


fn renew_certificates() -> Result<(), Box<dyn std::error::Error>> {

let mut firmware = Firmware::open()?;

// 기존 인증서 백업

let old_chain = firmware.pdh_cert_export()?;

save_certificate(&old_chain.pdh, "pdh_backup.der")?;

save_certificate(&old_chain.pek, "pek_backup.der")?;

// 새 키 생성

firmware.pek_generate()?;

firmware.pdh_generate()?;

// 새 인증서 체인 내보내기

let new_chain = firmware.pdh_cert_export()?;

println!("인증서 갱신 완료");

println!("이전 PDH ID: {:?}", old_chain.pdh.id());

println!("새 PDH ID: {:?}", new_chain.pdh.id());

Ok(())

}
```

2. 인증서 검증 및 신뢰 체인 구축

```rust
use sev::certs::sev::{Chain, Verifiable};
```

```rust
fn build_trust_chain() -> Result<(), Box<dyn std::error::Error>> {
// AMD 공식 인증서 로드 (내장 또는 다운로드)
let ark = load_amd_ark_certificate()?;
let ask = load_amd_ask_certificate()?;
// 플랫폼 인증서 로드
let mut firmware = Firmware::open()?;
let platform_chain = firmware.pdh_cert_export()?;
// 전체 신뢰 체인 구축
let ca_chain = ca::Chain { ark, ask };
let sev_chain = sev::Chain {
cek: platform_chain.cek,
pek: platform_chain.pek,
pdh: platform_chain.pdh,
};
let full_chain = Chain {
ca: ca_chain,
sev: sev_chain,
};
// 신뢰 체인 검증
match full_chain.verify() {
Ok(verified_cert) => {
println!("신뢰 체인 검증 성공");
```

```
println!("최종 검증된 인증서: {:?}", verified_cert.usage());
}
Err(e) => {
println!("신뢰 체인 검증 실패: {:?}", e);
return Err(e.into());
}
}
Ok(())
}

// AMD 공식 인증서 로드 (예시)
fn  load_amd_ark_certificate() -> Result<ca::Certificate, Box<dyn  std::error::E
// 실제로는 AMD에서 제공하는 공식 ARK 인증서를 로드
// 또는 내장된 인증서 사용
use sev::certs::sev::builtin;
Ok(builtin::naples::ark()?)
}

fn  load_amd_ask_certificate() -> Result<ca::Certificate, Box<dyn  std::error::E
// 실제로는 AMD에서 제공하는 공식 ASK 인증서를 로드
use sev::certs::sev::builtin;
Ok(builtin::naples::ask()?)
```

```
}
```

3. 인증서 만료 및 상태 관리

```
use std::time::{SystemTime, UNIX_EPOCH};


fn check_certificate_status() -> Result<(), Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
let chain = firmware.pdh_cert_export()?;
// 인증서 유효성 검사
println!("=== 인증서 상태 검사 ===");
// PDH 인증서 상태
check_individual_certificate(&chain.pdh, "PDH")?;
check_individual_certificate(&chain.pek, "PEK")?;
check_individual_certificate(&chain.oca, "OCA")?;
check_individual_certificate(&chain.cek, "CEK")?;
Ok(())
}


fn check_individual_certificate(cert: &Certificate, name: &str) -> Result<(), B
println!("--- {} 인증서 ---", name);
println!("사용 용도: {}", cert.usage());
// 인증서 기본 검증
```

```
if let Err(e) = cert.validate() {
println!("경고: {} 인증서 검증 실패: {:?}", name, e);
} else {
println!("{} 인증서 검증 성공", name);
}
Ok(())
}
```

보안 고려사항

1. 개인키 보호

```
use std::fs;
use std::os::unix::fs::PermissionsExt;

fn secure_key_storage(private_key: &PrivateKey<Usage>, filename: &str) -> Resul
// 개인키를 안전하게 저장
let key_data = private_key.encode_to_vec()?;
// 파일 생성 및 권한 설정 (소유자만 읽기/쓰기)
fs::write(filename, key_data)?;
let mut perms = fs::metadata(filename)?.permissions();
perms.set_mode(0o600); // rw-------
fs::set_permissions(filename, perms)?;
```

```
println!("개인키가 안전하게 저장되었습니다: {}", filename);
Ok(())
}
```

2. 인증서 검증 강화

```
fn enhanced_certificate_verification() -> Result<(), Box<dyn std::error::Error
let mut firmware = Firmware::open()?;
let chain = firmware.pdh_cert_export()?;
// 1. 기본 체인 검증
let verified_cert = chain.verify()?;
// 2. 추가 보안 검사
verify_certificate_policies(&chain)?;
verify_certificate_extensions(&chain)?;
verify_key_usage(&chain)?;
println!("강화된 인증서 검증 완료");
Ok(())
}

fn verify_certificate_policies(chain: &Chain) -> Result<(), Box<dyn std::error
// 인증서 정책 검증 로직
println!("인증서 정책 검증 중...");
Ok(())
```

```
}

fn verify_certificate_extensions(chain: &Chain) -> Result<(), Box<dyn std::err
// 인증서 확장 필드 검증 로직
println!("인증서 확장 필드 검증 중...");
Ok(())
}

fn verify_key_usage(chain: &Chain) -> Result<(), Box<dyn std::error::Error>> {
// 키 사용 용도 검증 로직
println!("키 사용 용도 검증 중...");
Ok(())
}
```

문제 해결

일반적인 인증서 오류

```
use sev::error::{UserApiError, CertError};

fn handle_certificate_errors() {
let mut firmware = Firmware::open().unwrap();
match firmware.pdh_cert_export() {
```

```
Ok(chain) => {
println!("인증서 체인 내보내기 성공");
}
Err(UserApiError::CertError(CertError::InvalidCertificate)) => {
println!("오류: 잘못된 인증서");
println!("해결: 인증서를 다시 생성하거나 올바른 인증서를 가져오세요");
}
Err(UserApiError::CertError(CertError::CertificateChainBroken)) => {
println!("오류: 인증서 체인이 손상됨");
println!("해결: 전체 인증서 체인을 다시 구축하세요");
}
Err(e) => {
println!("기타 오류: {:?}", e);
}
}
}
```

인증서 복구

```
fn recover_certificates() -> Result<(), Box<dyn std::error::Error>> {
let mut firmware = Firmware::open()?;
println!("인증서 복구 시작...");
// 1. 플랫폼 리셋
```

```
firmware.platform_reset()?;

println!("플랫폼 리셋 완료");

// 2. 새 키 생성

firmware.pek_generate()?;

firmware.pdh_generate()?;

println!("새 키 생성 완료");

// 3. 인증서 체인 확인

let  chain = firmware.pdh_cert_export()?;

match  chain.verify() {

Ok(_) => println!("인증서 복구 성공"),

Err(e) => println!("인증서 복구 실패: {:?}", e),

}

Ok(())

}
```

이 문서는 SEV 인증서 관리의 전체적인 개념과 실제 구현 방법을 제공합니다. 인증서 체인의 구조부터 실제 코드 예제까지 포함하여, 개발자가 SEV 환경에서 안전하고 효율적인 인증서 관리를 수행할 수 있도록 돕습니다.